

Implicit Return Type

Document: P0224R0
Date: 2016-02-05
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

This proposal recommends an enhancement to return type deduction to allow the return type of a function definition to be inferred from a previous declaration of the same.

(Note: references made to the existing draft standard are made against [N4567](#).)

Contents

- [Abstract](#)
- [Rationale](#)
- [Proposal](#)
- [Proposed Wording](#)
- [Interactions](#)
- [Discussion](#)
 - [Is this feature dangerous?](#)
 - [What about template return types?](#)
 - [Must the declaration providing the concrete type be the first declaration?](#)
- [Acknowledgments](#)
- [References](#)

Rationale

The origin for this idea relates to multiple return values, and a desire to use anonymous structs to implement the same. This notion will be further explored in a different paper (for which this proposal would be a prerequisite). The obvious problem with anonymous struct returns involves separating the declaration of such a function from its definition; namely, how does one repeat the name of the return type? This leads to a more interesting — and obvious in retrospect — question: since C++ functions (excluding templates) cannot be overloaded by their return value, **why is it necessary to repeat the return value at all?**

Even in the case of return types that can be named, it may be that repeating the type name is excessively verbose or otherwise undesirable. Some might even call this a violation of the [Don't Repeat Yourself](#) principle, similar to some of the issues that `auto` for variable declaration was introduced to solve. (On the flip side, one could see the ability to elide the return type as subject to many abuses, again in much the manner of `auto`. However, many language features can be abused; this should not prevent the addition of a feature that would provide an important benefit when used correctly.)

We would be remiss not to note that this is already possible in simple cases using `decltype` and a sample invocation of the function. However, while this may be adequate in simple cases, it is nevertheless needlessly verbose, and as the argument list grows longer, and/or gains arguments for which providing a legal value is non-trivial, it can quickly become unwieldy and unfeasible.

Proposal

Recent changes to the language have progressively relaxed the requirements for how return types are specified. We began with trailing return type specification, and have progressed to inferred return types in certain cases.

This proposal is to continue this direction by adding an additional use of `auto` as a return specifier, meaning "use the return type seen when this function was previously declared". The use of `auto` as the return type specifier, with no trailing return type, and for a function that has been previously declared with a known return type, shall instruct the compiler to define the function using the return type from the previous declaration.

Naturally, "previous declaration" here means a declaration having the same name and argument list. This, for example, would remain illegal:

```
int foo(int);
float foo(float);

auto foo(double input) // does not match any previous declaration
{
    ...
    return result;
}
```

Proposed Wording

(Proposed changes are specified relative [N4567](#).)

Add a new section to [dcl.spec.auto] (7.1.6.4) as follows:

When a function is declared or defined using `auto` for the return type, and a previous declaration or definition having a concrete return type exists, the return type shall be inferred to be the previously seen concrete type. [Example:

```
std::string f();
auto f(); // OK, return type is std::string
```

— end example]

Add a new section to [dcl.spec.auto] (7.1.6.4) as follows:

A template function redeclaration or specialization having a return type of `auto` shall match a previous declaration (or definition) if the first such declaration had a concrete return type. If the first such declaration also had a return type of `auto`, the declaration using return type deduction shall be matched instead. [Example:

```
template <typename T> T g(T t) { return t; } // #1
template auto g(float); // matches #1

template <typename T> auto g(T t) { return t; } // #2
template <typename T> T g(T t) { return t; }
template auto g(float); // matches #2
```

— end example]

Interactions

The three major C++ compilers (GCC, clang, MSVC) all presently reject the use of `auto` as a return type in the presence of a prior declaration of the same (non-template) function, even in the case that the deduced return type matches the prior declaration, although the resulting diagnostics vary (MSVC and clang refer to overloads differing only by return type, while GCC mentions an "ambiguating new declaration"). The case of template functions is more interesting: see [What about template return types?](#). We believe that only very obscure code would be affected by this change. (Affected code may further be impractical; that is, while such code could be written, it would not serve a useful purpose, and is thus unlikely to affect any code in actual use.)

This proposal does not make any changes to other existing language or library features. (Implementations, however, may wish to make use of it; doing so would be a non-breaking change, since the semantic meaning of the code would not be affected.)

Discussion

Is this feature dangerous?

Moritz Klammler noted that the effect of the following code may be unexpected:

```
float foo();  
auto foo() { return 42.0; }
```

If the declaration and definition are in different source files, it can appear that the return type of `foo()` is `double` (deduced), when under this proposal, it is in fact `float` (inferred).

In practice, we don't expect this to be a serious issue. The above example is overly simplified, to the point that we would not expect this feature to be used in such an instance (and even if so, compilers provide warnings for such conversions). In the same manner, modern C++ is more strict about potentially dangerous implicit conversions; in the case of modern `return` statements using type-elided braced initializers, an unsafe construct in a context like that presented would not be allowed.

Nevertheless, one way of addressing this problem would be to use some other syntax; either using something other than `auto` to request return type inference, or by adding (optional) syntax (e.g. a new contextual keyword) following the parameter list. While we do not oppose such alternatives, they delve into the well known challenge of Naming Things. A trailing contextual keyword — perhaps `inferred_return` — seems the only solution that does not risk invalidating existing code.

What about template return types?

In C++14, the following code is legal and produces two distinct templates:

```
template <class T> int foo();  
template <class T> auto foo();
```

This obviously conflicts with the proposed feature. After discussion on `std-proposals`, it was decided that the proposed feature should take precedence in this case. It should also be noted that it is unclear how, or even if, the second function can be invoked according to the current rules of the language. (To this end, it may be desirable to simply forbid the opposite ordering. However, we feel that this would be better addressed separately, perhaps even as a DR.)

Must the declaration providing the concrete type be the first declaration?

This question was originally brought up by Bengt Gustafsson. Specifically, for the sake of symmetry, it seems initially desirable to allow:

```
int foo(); // specified return type  
auto foo() { return 42; } // return type inferred from prior declaration  
  
auto bar(); // forward declaration, type not yet known  
int bar(); // specify the return type as 'int'  
auto bar() { return 0; } // return type inferred from prior declaration
```

To that end, earlier drafts of the proposal included the following proposed change to `[dcl.spec.auto]/13` (7.1.6.4.13):

Redeclarations or specializations of a function or function template with a declared return type that uses a placeholder type shall ~~also use that placeholder~~ use either that placeholder or a compatible concrete type, not a deduced type. If the return type has previously been deduced, a declaration using a concrete type shall use the deduced type. [Example:

```
auto f();  
auto f() { return 42; } // return type is int  
auto f(); // OK  
int f(); // error, cannot be overloaded with auto f()  
int f(); // OK, deduced type is also int  
decltype(auto) f(); // error, auto and decltype(auto) don't match  
  
auto f(int);  
int f(int); // OK, return type of f(int) is now int  
float f(int); // error, redeclared with different return type
```

However, upon further discussion, reservations were expressed, and the general consensus seems to be that it is okay for the first declaration to "set in stone" if the return type will be known (and possibly later inferred), or deduced. Accordingly, absent the above change:

```
auto bar();  
int bar(); // error, violates [dcl.spec.auto]/13  
auto bar() { return 0; } // okay, but return type is deduced, not inferred
```

Acknowledgments

We wish to thank everyone on the `std-proposals` forum, especially Bengt Gustafsson and Tim Song, for their valuable feedback and insights.

References

- [N4567](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf) Working Draft, Standard for Programming Language C++
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf>